



ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ
MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

BÁO CÁO BÀI TẬP LỚN

MÔN CẤU TRÚC RỜI RẠC

Topic: Traveling Salesman Problem

Trần Nguyễn Khánh-2411545

TP. HCM, 22-6-2025

MỤC LỤC

1	Giới thiệu	2
1.1	Tổng quan về chủ đề nghiên cứu	2
1.2	Sơ lược về 2 thuật toán được chọn	2
1.2.1	Dynamic Programming (thuật toán Held Karp)	2
1.2.2	Ant colony optimization (ACO)	3
2	Giải thích chi tiết thuật toán	3
2.1	Mã nguồn DP-Held Karp:	3
2.1.1	Thu thập các đỉnh duy nhất	6
2.1.2	Ánh xạ đỉnh sang chỉ số	7
2.1.3	Khởi tạo ma trận đồ thị	8
2.1.4	Khởi tạo bảng DP và parent	9
2.1.5	Hàm TSP sử dụng đệ quy với lambda	9
2.1.6	Tính chi phí tối thiểu	10
2.1.7	Xây dựng đường đi	11
2.2	Mã nguồn ACO:	11
2.2.1	Khai báo hằng số cho ACO	14
2.2.2	Hàm tạo số ngẫu nhiên đơn giản	15
2.2.3	Hàm lấy khoảng cách giữa hai đỉnh	16
2.2.4	Hàm chọn đỉnh kế tiếp dựa vào xác suất	16
2.2.5	Hàm chính ACO: Traveling	18
3	Tài liệu tham khảo	22

LỜI MỞ ĐẦU

- Bài toán Người bán hàng đi vòng quanh (TSP) là một bài toán kinh điển. Mục tiêu của TSP là tìm đường đi ngắn nhất qua tập thành phố, sao cho người bán hàng xuất phát từ điểm ban đầu, thăm mỗi thành phố một lần và trở về. Bài toán này NP-khó không có thuật toán chính xác đa thức cho mọi trường hợp. Nhiều phương pháp như Ant Colony Optimization và Genetic Algorithm tìm lời giải tối ưu gần đúng trong thời gian hợp lý. TSP có ứng dụng trong logistics và lập lịch công việc.

1 Giới thiệu

1.1 Tổng quan về chủ đề nghiên cứu

Bài toán Người bán hàng đi vòng (TSP) yêu cầu tìm chu trình ngắn nhất đi qua mọi đỉnh đúng một lần. Các phương pháp giải tiêu biểu gồm:

1. Brute-force.
2. Quy hoạch động Held–Karp với độ phức tạp $O(n^2 2^n)$.
3. Branch-and-bound và branch-and-cut kết hợp kỹ thuật cắt mặt phẳng.
4. Heuristic như nearest neighbor, 2-opt, 3-opt.
5. Thuật toán xấp xỉ Christofides đảm bảo sai lệch tối đa ≤ 1.5 lần tối ưu.
6. Metaheuristics như genetic algorithm, ant colony optimization, simulated annealing.

1.2 Sơ lược về 2 thuật toán được chọn

1.2.1 Dynamic Programming (thuật toán Held Karp)

Khi nào dùng Held–Karp?

- Khi $n \leq 20$ (đôi khi 22–24 nếu tối ưu tốt).
- Khi cần lời giải chính xác, không phải xấp xỉ.
- Khi bài toán có cấu trúc TSP rõ ràng.

⇒ Ở đây, em xin phép được sử dụng Held Karp cho phần submission của traveling 15_1, 15_2, 20_1 trên lms vì tính chính xác khi ít đỉnh.

1.2.2 Ant colony optimization (ACO)

Ưu điểm

- Dễ kết hợp heuristic cục bộ (ví dụ $1/d_{ij}$ trong TSP) để tăng tốc hội tụ.
- Khả năng “khám phá” và “khai thác” cân bằng nhờ trọng số pheromone (α) và heuristic (β).
- Tính mở rộng tốt với số đỉnh trung bình ($50 \leq n \leq 500$), cho tour gần tối ưu trong thời gian hợp lý.

Nhược điểm

- Nhiều tham số cần hiệu chỉnh (ví dụ $\alpha, \beta, \rho, Q, m$) để đạt hiệu năng tốt.
- Dễ bị kẹt tại cực tiểu cục bộ nếu tốc độ bay hơi (ρ) quá nhỏ.
- Chi phí tính toán cao với quần thể kiến lớn ($O(m \cdot n^2)$ mỗi vòng lặp).

⇒ Đối với các phần submission còn lại em xin được phép dùng giải thuật ACO để thực hiện vì số đỉnh lên nhiều và xấp xỉ là kết quả tương đối chấp nhận được

2 Giải thích chi tiết thuật toán

2.1 Mã nguồn DP-Held Karp:

```
1 std::string Traveling(int G[][3], int num_edges, char start) {
2     // Step 1: Collect unique vertices
3     std::vector<char> vertices;
4     for (int i = 0; i < num_edges; i++) {
5         char u = static_cast<char>(G[i][0]);
6         char v = static_cast<char>(G[i][1]);
7         if (std::find(vertices.begin(), vertices.end(), u) == vertices.end()
8             || std::find(vertices.begin(), vertices.end(), v) == vertices.end()) {
9             vertices.push_back(u);
10            vertices.push_back(v);
11        }
12    }
13    return TravelingDP(vertices, G, num_edges, start);
14 }
```

```

10     if (std::find(vertices.begin(), vertices.end(), v) == vertices.end
11         ()) {
12         vertices.push_back(v);
13     }
14     std::sort(vertices.begin(), vertices.end());
15     int n = vertices.size();
16
17     // Step 2: Map vertices to indices
18     std::vector<std::pair<char, int>> index_map;
19     for (int i = 0; i < n; i++) {
20         index_map.push_back(std::make_pair(vertices[i], i));
21     }
22
23     // Find index of starting vertex
24     int start_idx = -1;
25     for (const auto& p : index_map) {
26         if (p.first == start) {
27             start_idx = p.second;
28             break;
29         }
30     }
31     if (start_idx == -1) {
32         return "Error: Start vertex not found";
33     }
34
35     // Step 3: Initialize adjacency matrix
36     const int INF = 99999999;
37     std::vector<std::vector<int>> graph(n, std::vector<int>(n, INF));
38     for (int i = 0; i < n; i++) {
39         graph[i][i] = 0; // Distance from vertex to itself is 0
40     }
41     for (int i = 0; i < num_edges; i++) {
42         char u_char = static_cast<char>(G[i][0]);
43         char v_char = static_cast<char>(G[i][1]);
44         int u = -1, v = -1;
45         for (const auto& p : index_map) {
46             if (p.first == u_char) u = p.second;
47             if (p.first == v_char) v = p.second;
48         }
49         if (u != -1 && v != -1) {
50             graph[u][v] = G[i][2]; // Assign edge weight
51         }
52     }
53
54     // Step 4: Initialize DP table and parent array
55     int max_mask = 1 << n;
56     std::vector<std::vector<int>> dp(max_mask, std::vector<int>(n, -1));
57     std::vector<std::vector<int>> parent(max_mask, std::vector<int>(n,
    -1));

```

```

58
59 // Step 5: TSP function using recursion with lambda
60 auto tsp = [&](auto&& self, int mask, int pos) -> int {
61     if (mask == (1 << n) - 1) { // Visited all vertices
62         return graph[pos][start_idx]; // Return to starting vertex
63     }
64     if (dp[mask][pos] != -1) { // Already computed
65         return dp[mask][pos];
66     }
67     int ans = INF;
68     for (int next = 0; next < n; next++) {
69         if ((mask & (1 << next)) == 0 && graph[pos][next] != INF) {
70             int new_mask = mask | (1 << next);
71             int cost = graph[pos][next] + self(self, new_mask, next);
72             if (cost < ans) {
73                 ans = cost;
74                 parent[mask][pos] = next; // Store next vertex
75             }
76         }
77     }
78     dp[mask][pos] = ans;
79     return ans;
80 };
81
82 // Step 6: Calculate minimum cost
83 int min_cost = tsp(tsp, 1 << start_idx, start_idx);
84
85 // Step 7: Construct path
86 std::string path;
87 int mask = 1 << start_idx;
88 int pos = start_idx;
89 while (true) {
90     path += vertices[pos];
91     path += " ";
92     int next = parent[mask][pos];
93     if (next == -1) break;
94     mask |= (1 << next);
95     pos = next;
96 }
97 path += start; // Return to starting vertex
98
99 return path;
100 }

```

Listing 1: DP and Held Karp algorithm

Tóm tắt hàm Traveling thực hiện các bước sau:

1. Thu thập các đỉnh duy nhất từ danh sách cạnh.

2. Ánh xạ đỉnh thành chỉ số.
3. Xây dựng ma trận đồ thị.
4. Khởi tạo bảng DP và parent.
5. Sử dụng hàm đệ quy `tsp` để tìm chi phí tối thiểu.
6. Tính chi phí tối thiểu từ đỉnh xuất phát.
7. Xây dựng và trả về đường đi dưới dạng chuỗi.

```
1 std::string Traveling(int G[][3], int num_edges, char start) {
```

Đây là khai báo hàm `Traveling`:

Tham số:

- `G[][3]`: Một mảng 2 chiều kiểu `int`, đại diện cho danh sách các cạnh của đồ thị. Mỗi hàng chứa 3 giá trị: đỉnh nguồn, đỉnh đích, và trọng số của cạnh.
- `num_edges`: Số lượng cạnh trong đồ thị.
- `start`: Ký tự đại diện cho đỉnh xuất phát.

Kiểu trả về: `std::string`, là chuỗi biểu diễn đường đi tối ưu.

2.1.1 Thu thập các đỉnh duy nhất

```
1 std::vector<char> vertices;
```

Khởi tạo một vector `vertices` kiểu `char` để lưu trữ các đỉnh duy nhất trong đồ thị.

```
1 for (int i = 0; i < num_edges; i++) {
2     char u = static_cast<char>(G[i][0]);
3     char v = static_cast<char>(G[i][1]);
```

Vòng lặp duyệt qua từng cạnh trong `G`:

- `G[i][0]`: Đỉnh nguồn của cạnh thứ `i`, được chuyển đổi từ `int` sang `char` và lưu vào `u`.
- `G[i][1]`: Đỉnh đích của cạnh thứ `i`, được chuyển đổi từ `int` sang `char` và lưu vào `v`.

```

1  if (std::find(vertices.begin(), vertices.end(), u) == vertices.end())
2      {
3          vertices.push_back(u);
4      }
5  if (std::find(vertices.begin(), vertices.end(), v) == vertices.end())
6      {
7          vertices.push_back(v);
8      }

```

Kiểm tra xem `u` và `v` đã có trong `vertices` chưa:

- `std::find` trả về `vertices.end()` nếu không tìm thấy phần tử.
- Nếu `u` hoặc `v` chưa có, thêm chúng vào `vertices` bằng `push_back`.

```

1 std::sort(vertices.begin(), vertices.end());
2 int n = vertices.size();

```

- Sắp xếp `vertices` theo thứ tự tăng dần để đảm bảo tính nhất quán.
- `n` là số lượng đỉnh duy nhất trong đồ thị.

2.1.2 Ánh xạ đỉnh sang chỉ số

```

1 std::vector<std::pair<char, int>> index_map;
2 for (int i = 0; i < n; i++) {
3     index_map.push_back(std::make_pair(vertices[i], i));
4 }

```

Tạo vector `index_map` chứa các cặp <ký tự đỉnh, chỉ số>:

- Mỗi đỉnh trong `vertices` được gán một chỉ số từ 0 đến $n - 1$.
- Ví dụ: Nếu `vertices = {'A', 'B', 'C'}`, thì `index_map = {{'A', 0}, {'B', 1}, {'C', 2}}`.

```

1 int start_idx = -1;
2 for (const auto& p : index_map) {
3     if (p.first == start) {
4         start_idx = p.second;
5         break;
6     }
7 }

```

Tìm chỉ số của đỉnh xuất phát `start` trong `index_map`:

- `p.first` là ký tự đỉnh, `p.second` là chỉ số.

- Nếu tìm thấy, gán chỉ số vào `start_idx` và thoát vòng lặp.

```

1 if (start_idx == -1) {
2     return "Error: Start vertex not found";
3 }

```

⇒ Nếu không tìm thấy `start` (`start_idx` vẫn là -1), trả về thông báo lỗi.

2.1.3 Khởi tạo ma trận đồ thị

```

1 const int INF = 999999999;
2 std::vector<std::vector<int>>> graph(n, std::vector<int>(n, INF));

```

- Định nghĩa `INF` là một giá trị lớn biểu thị khoảng cách vô cực.
- Tạo ma trận `graph` kích thước $n \times n$, khởi tạo tất cả phần tử bằng `INF`.

```

1 for (int i = 0; i < n; i++) {
2     graph[i][i] = 0;
3 }

```

- Đặt các phần tử trên đường chéo chính của `graph` bằng 0, vì khoảng cách từ một đỉnh đến chính nó là 0.

```

1 for (int i = 0; i < num_edges; i++) {
2     char u_char = static_cast<char>(G[i][0]);
3     char v_char = static_cast<char>(G[i][1]);
4     int u = -1, v = -1;
5     for (const auto& p : index_map) {
6         if (p.first == u_char) u = p.second;
7         if (p.first == v_char) v = p.second;
8     }
9     if (u != -1 && v != -1) {
10        graph[u][v] = G[i][2];
11    }
12 }

```

- Duyệt qua từng cạnh trong `G`:
 - Lấy đỉnh nguồn `u_char` và đỉnh đích `v_char` từ `G[i][0]` và `G[i][1]`.
 - Tìm chỉ số `u` và `v` tương ứng trong `index_map`.
- Nếu cả `u` và `v` hợp lệ, gán trọng số của cạnh (`G[i][2]`) vào `graph[u][v]`.

2.1.4 Khởi tạo bảng DP và parent

```
1 int max_mask = 1 << n;  
2 std::vector<std::vector<int>> dp(max_mask, std::vector<int>(n, -1));  
3 std::vector<std::vector<int>> parent(max_mask, std::vector<int>(n, -1));
```

- `max_mask = 1 « n`: Đây là 2^n , biểu diễn tất cả các trạng thái có thể của tập hợp đỉnh (đã thăm hay chưa thăm).
- `dp`: Bảng lưu chi phí tối thiểu cho từng trạng thái (`mask`) và vị trí hiện tại (`pos`), được khởi tạo bằng -1 .
- `parent`: Bảng lưu đỉnh tiếp theo trong đường đi tối ưu, cũng được khởi tạo bằng -1 .

2.1.5 Hàm TSP sử dụng đệ quy với lambda

```
1 auto tsp = [&](auto&& self, int mask, int pos) -> int {  
2     if (mask == (1 << n) - 1) {  
3         return graph[pos][start_idx];  
4     }  
5     if (dp[mask][pos] != -1) {  
6         return dp[mask][pos];  
7     }  
8     int ans = INF;  
9     for (int next = 0; next < n; next++) {  
10        if ((mask & (1 << next)) == 0 && graph[pos][next] != INF) {  
11            int new_mask = mask | (1 << next);  
12            int cost = graph[pos][next] + self(self, new_mask, next);  
13            if (cost < ans) {  
14                ans = cost;  
15                parent[mask][pos] = next;  
16            }  
17        }  
18    }  
19    dp[mask][pos] = ans;  
20    return ans;  
21 };
```

- `auto tsp = [&](auto&& self, int mask, int pos) -> int`: Định nghĩa hàm đệ quy `tsp` sử dụng lambda:
 - `self`: Tham chiếu đến chính hàm lambda để thực hiện đệ quy.
 - `mask`: Bitmask biểu diễn tập hợp các đỉnh đã thăm.
 - `pos`: Chỉ số của đỉnh hiện tại.

- **Trường hợp cơ sở:** Nếu `mask == (1 << n) - 1` (tất cả các bit đều là 1), tức là đã thăm hết các đỉnh, trả về chi phí từ `pos` về `start_idx`.
- Nếu `dp[mask][pos] != -1`, trả về giá trị đã được tính trước đó trong bảng `dp`.
- Khởi tạo `ans = INF` để tìm chi phí tối thiểu.
- Duyệt qua tất cả các đỉnh `next`:
 - `(mask & (1 << next)) == 0`: Kiểm tra xem đỉnh `next` chưa được thăm.
 - `graph[pos][next] != INF`: Kiểm tra xem có cạnh từ `pos` đến `next`.
 - `new_mask = mask | (1 << next)`: Đánh dấu đỉnh `next` là đã thăm bằng phép OR bitwise.
 - `cost = graph[pos][next] + self(self, new_mask, next)`: Tính tổng chi phí từ `pos` đến `next` cộng với chi phí tối thiểu từ `next` đến hết (gọi đệ quy).
 - Nếu `cost < ans`, cập nhật `ans` và lưu `next` vào `parent[mask][pos]` để theo dõi đường đi.
- Lưu giá trị `ans` vào `dp[mask][pos]` và trả về.

2.1.6 Tính chi phí tối thiểu

```
1 int min_cost = tsp(tsp, 1 << start_idx, start_idx);
```

- Gọi hàm `tsp` với:
 - `mask = 1 << start_idx`: Bitmask ban đầu, chỉ đỉnh xuất phát được đánh dấu là đã thăm.
 - `pos = start_idx`: Bắt đầu từ đỉnh xuất phát.
- `min_cost` lưu trữ chi phí tối thiểu của toàn bộ đường đi, mặc dù không sử dụng trực tiếp trong kết quả cuối cùng.

2.1.7 Xây dựng đường đi

```
1 std::string path;
2 int mask = 1 << start_idx;
3 int pos = start_idx;
4 while (true) {
5     path += vertices[pos];
6     path += " ";
7     int next = parent[mask][pos];
8     if (next == -1) break;
9     mask |= (1 << next);
10    pos = next;
11 }
12 path += start;
13 return path;
```

- Khởi tạo path là chuỗi rỗng để lưu đường đi.
- mask và pos bắt đầu từ đỉnh xuất phát (start_idx).
- Vòng lặp xây dựng đường đi:
 - Thêm ký tự của đỉnh hiện tại (vertices[pos]) và một khoảng trắng vào path.
 - Lấy đỉnh tiếp theo từ parent[mask][pos].
 - Nếu next == -1, thoát vòng lặp (đã đến cuối đường đi).
 - Cập nhật mask bằng phép OR bitwise với 1 « next và chuyển pos sang next.
- Thêm đỉnh xuất phát (start) vào cuối path để hoàn thành chu trình.
- Trả về path là chuỗi biểu diễn đường đi tối ưu.

2.2 Mã nguồn ACO:

```
1 // Constants for ACO
2 const int MAX_ITERATIONS = 100;
3 const int NUM_ANTS = 10;
4 const double ALPHA = 1.0;
5 const double BETA = 2.0;
6 const double RHO = 0.1;
7 const double Q = 100.0;
8 const double TAU0 = 1.0;
9
```

```

10 // Simple random number generator
11 double simpleRand(int& seed) {
12     seed = (seed * 1103515245 + 12345) & 0x7fffffff;
13     return static_cast<double>(seed) / 0x7fffffff;
14 }
15
16 // Get distance between two vertices
17 double getDistance(char u, char v, int edgeList[][3], int numEdge) {
18     for (int i = 0; i < numEdge; ++i) {
19         if (static_cast<char>(edgeList[i][0]) == u && static_cast<char>(
20             edgeList[i][1]) == v) {
21             return edgeList[i][2];
22         }
23     }
24     return 1e9; // Large value if no edge exists
25 }
26
27 // Select next vertex based on probability
28 char selectNextVertex(char current, const std::vector<char>& unvisited,
29     const std::vector<std::pair<std::pair<char, char>,
30         double>>& pheromone,
31     int edgeList[][3], int numEdge, int& seed) {
32     double total = 0.0;
33     std::vector<std::pair<char, double>> probabilities;
34     for (char next : unvisited) {
35         for (const auto& p : pheromone) {
36             if (p.first.first == current && p.first.second == next) {
37                 double tau = p.second;
38                 double eta = 1.0 / getDistance(current, next, edgeList,
39                     numEdge);
40                 double value = pow(tau, ALPHA) * pow(eta, BETA);
41                 probabilities.push_back({next, value});
42                 total += value;
43                 break;
44             }
45         }
46     }
47     if (total == 0.0 && !unvisited.empty()) return unvisited[0];
48     double r = simpleRand(seed) * total;
49     double sum = 0.0;
50     for (const auto& prob : probabilities) {
51         sum += prob.second;
52         if (sum >= r) return prob.first;
53     }
54     return unvisited[0];
55 }
56
57 // Main ACO function
58 string Traveling(int edgeList[][3], int numEdge, char start) {
59     int seed = 123456789; // Fixed seed for pseudo-randomness

```

```

57 // Step 1: Identify all vertices
58 std::vector<char> vertices;
59 for (int i = 0; i < numEdge; ++i) {
60     char u = static_cast<char>(edgeList[i][0]);
61     char v = static_cast<char>(edgeList[i][1]);
62     if (std::find(vertices.begin(), vertices.end(), u) == vertices.end()
        ()) vertices.push_back(u);
63     if (std::find(vertices.begin(), vertices.end(), v) == vertices.end()
        ()) vertices.push_back(v);
64 }
65 int numVertices = vertices.size();
66 // Step 2: Initialize pheromone for existing edges
67 std::vector<std::pair<std::pair<char, char>, double>> pheromone;
68 for (int i = 0; i < numEdge; ++i) {
69     char u = static_cast<char>(edgeList[i][0]);
70     char v = static_cast<char>(edgeList[i][1]);
71     pheromone.push_back({{u, v}, TAU0});
72 }
73 std::vector<char> bestPath;
74 double bestLength = 1e9;
75 // Step 3: ACO iterations
76 for (int iter = 0; iter < MAX_ITERATIONS; ++iter) {
77     std::vector<std::vector<char>> antPaths(NUM_ANTS);
78     std::vector<double> antLengths(NUM_ANTS, 0.0);
79     for (int ant = 0; ant < NUM_ANTS; ++ant) {
80         std::vector<char>& path = antPaths[ant];
81         std::vector<char> unvisited = vertices;
82         char current = start;
83         path.push_back(current);
84         unvisited.erase(std::find(unvisited.begin(), unvisited.end(),
            current));
85         // Build path
86         while (!unvisited.empty()) {
87             char next = selectNextVertex(current, unvisited, pheromone
                , edgeList, numEdge, seed);
88             path.push_back(next);
89             antLengths[ant] += getDistance(current, next, edgeList,
                numEdge);
90             current = next;
91             unvisited.erase(std::find(unvisited.begin(), unvisited.end()
                (), next));
92         }
93         // Return to start
94         path.push_back(start);
95         antLengths[ant] += getDistance(current, start, edgeList,
            numEdge);
96         // Update best path
97         if (antLengths[ant] < bestLength) {
98             bestLength = antLengths[ant];
99             bestPath = path;

```

```

100     }
101 }
102 // Update pheromone
103 for (auto& p : pheromone) {
104     p.second *= (1.0 - RHO); // Evaporation
105 }
106 for (int ant = 0; ant < NUM_ANTS; ++ant) {
107     for (size_t i = 0; i < antPaths[ant].size() - 1; ++i) {
108         char u = antPaths[ant][i];
109         char v = antPaths[ant][i + 1];
110         for (auto& p : pheromone) {
111             if (p.first.first == u && p.first.second == v) {
112                 p.second += Q / antLengths[ant];
113                 break;
114             }
115         }
116     }
117 }
118 }
119 // Step 4: Construct the result string
120 std::string result;
121 for (size_t i = 0; i < bestPath.size(); ++i) {
122     result += bestPath[i];
123     if (i < bestPath.size() - 1) result += " ";
124 }
125 return result;
126 }

```

Các công đoạn chính:

1. Xác định các đỉnh (Identify all vertices):

Duyệt qua danh sách cạnh để thu thập tất cả các đỉnh có mặt.

2. Khởi tạo pheromone (Initialize pheromone):

Gán giá trị pheromone ban đầu cho tất cả các cạnh có tồn tại.

3. Thực hiện quá trình lặp ACO (ACO iterations):

Lặp qua số lượng vòng lặp xác định, với mỗi "kiến" xây dựng đường đi, tính tổng khoảng cách và cập nhật pheromone theo hiệu quả của đường đi.

4. Xây dựng chuỗi kết quả (Construct the result string):

Nối các đỉnh của đường đi tốt nhất thành chuỗi để trả về kết quả.

2.2.1 Khai báo hằng số cho ACO

```

1 // Constants for ACO
2 const int MAX_ITERATIONS = 100;
3 const int NUM_ANTS = 10;
4 const double ALPHA = 1.0;
5 const double BETA = 2.0;
6 const double RHO = 0.1;
7 const double Q = 100.0;
8 const double TAU0 = 1.0;

```

Giải thích:

- MAX_ITERATIONS: Số vòng lặp tối đa mà thuật toán sẽ chạy (100 lần).
- NUM_ANTS: Số lượng “kiến” (agents) tham gia xây dựng các đường đi.
- ALPHA: Hệ số điều chỉnh tầm quan trọng của lượng pheromone khi lựa chọn.
- BETA: Hệ số điều chỉnh tầm quan trọng của giá trị heuristic (độ thuận lợi) khi chọn đường.
- RHO: Tỷ lệ bay hơi của pheromone. Sau mỗi vòng lặp, lượng pheromone được giảm theo tỷ lệ này.
- Q: Hằng số dùng để định lượng lượng pheromone được bổ sung sau khi một đường đi hoàn thành.
- TAU0: Giá trị khởi tạo cho pheromone trên mỗi cạnh.

2.2.2 Hàm tạo số ngẫu nhiên đơn giản

```

1 // Simple random number generator
2 double simpleRand(int& seed) {
3     seed = (seed * 1103515245 + 12345) & 0x7fffffff;
4     return static_cast<double>(seed) / 0x7fffffff;
5 }

```

Giải thích:

- Dòng `seed = (seed * 1103515245 + 12345) & 0x7fffffff;` cập nhật biến `seed` theo công thức của hàm sinh số ngẫu nhiên loại Linear Congruential Generator. Phép `& 0x7fffffff` đảm bảo kết quả luôn là số dương.

- Dòng `return static_cast<double>(seed) / 0x7fffffff;` chuyển `seed` sang kiểu số thực và chia cho giá trị tối đa nhằm chuẩn hóa phạm vi về $[0,1]$.

2.2.3 Hàm lấy khoảng cách giữa hai đỉnh

```

1 // Get distance between two vertices
2 double getDistance(char u, char v, int edgeList[][3], int numEdge) {
3     for (int i = 0; i < numEdge; ++i) {
4         if (static_cast<char>(edgeList[i][0]) == u && static_cast<char>(
5             edgeList[i][1]) == v) {
6             return edgeList[i][2];
7         }
8     }
9     return 1e9; // Large value if no edge exists
10 }

```

Giải thích:

- Hàm nhận các đối số: `u`, `v` là hai đỉnh cần tính khoảng cách, `edgeList` chứa thông tin các cạnh (đỉnh nguồn, đỉnh đích, trọng số) và `numEdge` là số cạnh.
- Vòng lặp `for` duyệt qua tất cả các cạnh:
 - Nếu tìm thấy một cạnh sao cho đỉnh nguồn bằng `u` và đỉnh đích bằng `v`, trả về trọng số của cạnh đó.
- Nếu không tìm thấy cạnh, hàm trả về giá trị rất lớn (10^9) như một cách biểu thị khoảng cách vô cực.

2.2.4 Hàm chọn đỉnh kế tiếp dựa vào xác suất

```

1 // Select next vertex based on probability
2 char selectNextVertex(char current, const std::vector<char>& unvisited,
3     const std::vector<std::pair<std::pair<char, char>,
4         double>>& pheromone,
5     int edgeList[][3], int numEdge, int& seed) {
6     double total = 0.0;
7     std::vector<std::pair<char, double>> probabilities;
8
9     for (char next : unvisited) {
10         for (const auto& p : pheromone) {
11             if (p.first.first == current && p.first.second == next) {
12                 double tau = p.second;
13                 double eta = 1.0 / getDistance(current, next, edgeList,
14                     numEdge);

```

```

13         double value = pow(tau, ALPHA) * pow(eta, BETA);
14         probabilities.push_back({next, value});
15         total += value;
16         break;
17     }
18 }
19 }
20
21 if (total == 0.0 && !unvisited.empty()) return unvisited[0];
22
23 double r = simpleRand(seed) * total;
24 double sum = 0.0;
25 for (const auto& prob : probabilities) {
26     sum += prob.second;
27     if (sum >= r) return prob.first;
28 }
29 return unvisited[0];
30 }

```

Giải thích:

1. Khởi tạo biến `total` để tính tổng giá trị xác suất và một vector `probabilities` để lưu các cặp {đỉnh, giá trị}.
2. Duyệt qua mỗi đỉnh `next` trong danh sách `unvisited`.
3. Với mỗi `next`, duyệt qua các phần tử trong vector `pheromone`:
 - Nếu tìm thấy cạnh có đỉnh nguồn bằng `current` và đỉnh đích bằng `next`, thực hiện:
 - Lấy giá trị pheromone τ từ `p.second`.
 - Tính giá trị heuristic $\eta = \frac{1}{\text{getDistance}(\text{current}, \text{next}, \text{edgeList}, \text{numEdge})}$.
 - Tính `value` theo công thức: $\text{value} = \tau^{ALPHA} \times \eta^{BETA}$.
 - Thêm cặp `{next, value}` vào vector `probabilities` và cộng `value` vào `total`.
 - Dùng `break` để dừng vòng lặp nội bộ khi tìm được cạnh phù hợp.
4. Nếu `total` bằng 0 nhưng danh sách `unvisited` không rỗng, trả về đỉnh đầu tiên trong `unvisited` như dự phòng.
5. Sinh một số ngẫu nhiên r thông qua hàm `simpleRand(seed)` nhân với `total`.
6. Duyệt qua vector `probabilities`; tích lũy giá trị xác suất vào biến `sum`.

7. Trả về đỉnh khi sum vượt quá hoặc bằng r .
8. Nếu không chọn được đỉnh nào (trường hợp hiếm), trả về đỉnh đầu tiên của `unvisited`.

2.2.5 Hàm chính ACO: Traveling

```
1 // Main ACO function
2 string Traveling(int edgeList[][3], int numEdge, char start) {
3     int seed = 123456789; // Fixed seed for pseudo-randomness
```

Giải thích:

- Hàm `Traveling` nhận vào ma trận cạnh `edgeList`, số cạnh `numEdge` và đỉnh xuất phát `start`.
- Biến `seed` được khởi tạo với giá trị cố định để đảm bảo tính tái lập cho các số ngẫu nhiên.

Bước 1: Xác định các đỉnh

```
1 // Step 1: Identify all vertices
2 std::vector<char> vertices;
3 for (int i = 0; i < numEdge; ++i) {
4     char u = static_cast<char>(edgeList[i][0]);
5     char v = static_cast<char>(edgeList[i][1]);
6     if (std::find(vertices.begin(), vertices.end(), u) == vertices.end()
7         ) vertices.push_back(u);
8     if (std::find(vertices.begin(), vertices.end(), v) == vertices.end()
9         ) vertices.push_back(v);
10 }
11 int numVertices = vertices.size();
```

Giải thích:

1. Khởi tạo vector `vertices` để lưu các đỉnh duy nhất từ danh sách cạnh.
2. Với mỗi cạnh, lấy đỉnh nguồn `u` và đỉnh đích `v` (sau khi chuyển từ `int` sang `char`).
3. Kiểm tra nếu `u` hoặc `v` chưa có trong vector, thêm chúng vào.
4. Xác định số lượng đỉnh duy nhất và lưu vào biến `numVertices`.

Bước 2: Khởi tạo lượng pheromone

```
1 // Step 2: Initialize pheromone for existing edges
2 std::vector<std::pair<std::pair<char, char>, double>> pheromone;
3 for (int i = 0; i < numEdge; ++i) {
4     char u = static_cast<char>(edgeList[i][0]);
5     char v = static_cast<char>(edgeList[i][1]);
6     pheromone.push_back({{u, v}, TAU0});
7 }
```

Giải thích:

1. Tạo vector **pheromone** lưu trữ các cặp gồm thông tin cạnh (đỉnh nguồn, đỉnh đích) và giá trị pheromone khởi tạo ($TAU0$).
2. Duyệt qua từng cạnh để đưa mỗi cặp $\{u, v\}$ vào vector với giá trị mặc định.

Bước 3: Vòng lặp ACO (Tìm đường đi tối ưu)

```
1 std::vector<char> bestPath;
2 double bestLength = 1e9;
3
4 // Step 3: ACO iterations
5 for (int iter = 0; iter < MAX_ITERATIONS; ++iter) {
6     std::vector<std::vector<char>> antPaths(NUM_ANTS);
7     std::vector<double> antLengths(NUM_ANTS, 0.0);
8
9     for (int ant = 0; ant < NUM_ANTS; ++ant) {
10         std::vector<char>& path = antPaths[ant];
11         std::vector<char> unvisited = vertices;
12         char current = start;
13         path.push_back(current);
14         unvisited.erase(std::find(unvisited.begin(), unvisited.end(),
15                                   current));
16
17         // Build path
18         while (!unvisited.empty()) {
19             char next = selectNextVertex(current, unvisited, pheromone,
20                                           edgeList, numEdge, seed);
21             path.push_back(next);
22             antLengths[ant] += getDistance(current, next, edgeList,
23                                             numEdge);
24             current = next;
25             unvisited.erase(std::find(unvisited.begin(), unvisited.end(),
26                                       next));
27         }
28
29         // Return to start
30         path.push_back(start);
```

```

26         antLengths[ant] += getDistance(current, start, edgeList,
27                                         numEdge);
28
29         // Update best path
30         if (antLengths[ant] < bestLength) {
31             bestLength = antLengths[ant];
32             bestPath = path;
33         }
34     }

```

Giải thích:

1. Khởi tạo vector **bestPath** và biến **bestLength** (với giá trị rất lớn ban đầu) để lưu đường đi tối ưu.
2. Trong mỗi vòng lặp ACO (theo số vòng **MAX_ITERATIONS**):
 - Khởi tạo **antPaths** để lưu đường đi của từng kiến và **antLengths** để lưu tổng khoảng cách của đường đi tương ứng.
 - Với mỗi kiến:
 - Lấy tham chiếu **path** từ **antPaths[ant]** và sao chép danh sách các đỉnh vào **unvisited**.
 - Đặt đỉnh xuất phát **start** làm điểm khởi đầu, thêm vào **path** và loại bỏ khỏi **unvisited**.
 - Vòng lặp **while** sử dụng để xây dựng đường đi:
 - Gọi hàm **selectNextVertex** để chọn đỉnh kế tiếp dựa trên xác suất (lượng pheromone và khoảng cách).
 - Thêm đỉnh **next** vào **path**.
 - Cộng khoảng cách từ đỉnh **current** đến đỉnh **next** vào **antLengths[ant]**.
 - Cập nhật **current** thành **next** và loại bỏ **next** khỏi danh sách **unvisited**.
 - Sau khi kiến đã thăm hết các đỉnh, thêm đỉnh xuất phát vào cuối đường đi để tạo thành chu trình và cập nhật thêm khoảng cách từ đỉnh cuối về đỉnh xuất phát.
 - Cập nhật **bestPath** và **bestLength** nếu đường đi hiện tại có tổng khoảng cách nhỏ hơn.

```

1         // Update pheromone
2         for (auto& p : pheromone) {
3             p.second *= (1.0 - RHO); // Evaporation
4         }
5         for (int ant = 0; ant < NUM_ANTS; ++ant) {

```

```

6         for (size_t i = 0; i < antPaths[ant].size() - 1; ++i) {
7             char u = antPaths[ant][i];
8             char v = antPaths[ant][i + 1];
9             for (auto& p : pheromone) {
10                if (p.first.first == u && p.first.second == v) {
11                    p.second += Q / antLengths[ant];
12                    break;
13                }
14            }
15        }
16    }
17 }

```

Giải thích cập nhật pheromone:

1. Sau mỗi vòng lặp, cập nhật pheromone cho mọi cạnh:

- Giảm pheromone trên mỗi cạnh theo tỷ lệ bay hơi RHO .

2. Sau đó, với mỗi kiến:

- Duyệt qua các cặp đỉnh liên tiếp trong đường đi.
- Tìm cạnh tương ứng trong danh sách **pheromone** và tăng lượng pheromone bằng $\frac{Q}{\text{antLengths}[\text{ant}]}$, giúp khuyến khích các đường đi có tổng khoảng cách nhỏ.

Bước 4: Xây dựng chuỗi kết quả

```

1 // Step 4: Construct the result string
2 std::string result;
3 for (size_t i = 0; i < bestPath.size(); ++i) {
4     result += bestPath[i];
5     if (i < bestPath.size() - 1) result += " ";
6 }
7 return result;
8 }

```

Giải thích:

- Khởi tạo chuỗi **result** rỗng.
- Duyệt qua vector **bestPath** (đường đi tối ưu) và nối các ký tự đỉnh vào chuỗi, thêm dấu cách giữa các đỉnh.
- Trả về chuỗi **result** chứa đường đi tối ưu.

3 Tài liệu tham khảo

1. Sách DISCRETE MATHEMATICS AND ITS APPLICATIONS, EIGHTH EDITION-Chương 10-Graph, tác giả: Kenneth H. Rosen
2. Video của Abdul Bari về ứng dụng của Dynamic Programming trong TSP: <https://www.youtube.com/watch?v=XaXsJJh-Q5Y&t=790s>.
3. Video của Reducible về các giải thuật để giải bài toán traveling salesman: <https://www.youtube.com/watch?v=GiDsjiBOVoA&t=243s>.
4. Video của Project Fi: <https://www.youtube.com/watch?v=Kahrabp8WZ0>.
5. Video của HK Lam: <https://www.youtube.com/watch?v=jNd7QJQH-kk>.